Square-Root Circuit Bounds for Iterated Functions

Samuel Schlesinger

October 15, 2026

Preliminary draft

Abstract

Let $F : \{0,1\}^n \to \{0,1\}^n$ be computed by a Boolean circuit of size s, and write $A = s + n^3$. We prove that every fixed iterate F^T, for an arbitrary integer $T \geq 0$, has a bounded-fan-in Boolean circuit of size

$$O\Big(A + \sqrt{n^3 2^n A}\Big).$$

The circuit is nonuniform in F and T, but the bound is independent of the magnitude and bit length of T. No injectivity assumption is required. Consequently, all fixed iterates of a polynomial-size update rule have circuits of size $2^{n/2+O(\log n)}$. The same exponential bound holds when an n-bit iteration count is part of the input. We also give a batched construction that shares preprocessing across independent trajectories.

The proof combines sparse checkpoints in a functional graph with a forward-only correction identity and batched dictionary lookup implemented by sorting networks. All lookup and selection gates are included in the size bound. The theorem recovers square-root-size circuits for permutation inversion, consistently with possible one-wayness against polynomial-size adversaries. Finally, for even n we construct an explicit linear-size permutation on n bits for which some fixed power requires $\Omega(2^{n/2}/n)$ gates. Such hard powers necessarily have binary exponents of length $\Omega(2^{n/2}/n^2)$. The counting argument neither identifies an explicit hard power nor establishes one-way functions. A Lean formalization checks the fixed-power tradeoff, its invariant-set and image-sensitive extensions, and transport to encoded state types in arbitrary universes. We also derive cycle labels, reachability, and first hitting times; these navigation results are not yet mechanized.

Keywords. Boolean circuit complexity; function iteration; functional graphs; nonuniform computation; sorting networks; permutation inversion; one-way functions.

1 Introduction

A simple transition rule can describe a computation of enormous duration. How complicated can its final-state map be as a Boolean function of the initial state? We study this question for a fixed map $F : \{0,1\}^n \to \{0,1\}^n$. The computational resource is the total number of Boolean gates, rather than the number of evaluations of F, the number of memory probes, or the time of a random-access algorithm.

Let $N = 2^n$ and suppose that F has a circuit of size s. Directly unrolling T applications gives a circuit of size at most Ts for F^T. Alternatively, hardwiring a complete truth table and testing the input against all N states gives an $O(nN)$-gate circuit, independently of T. The latter bound

ignores the possibility that F itself is much simpler than an arbitrary map on its state space. Our main result exploits that possibility.

Theorem 1.1 (Fixed-power tradeoff)**.** *Let $n \geq 1$, let $F : \{0,1\}^n \to \{0,1\}^n$ satisfy $\mathrm{C}(F) \leq s$, and put $N = 2^n$ and $A = s + n^3$. For every integer $T \geq 0$ and every integer $1 \leq L \leq N$,*

$$\mathrm{C}(F^T) = O\Big(Ls + n^3\left(L + \frac{N}{L}\right)\Big). \tag{1.1}$$

In particular,

$$\mathrm{C}(F^T) = O\Big(A + \sqrt{n^3 NA}\Big). \tag{1.2}$$

The implicit constants are absolute. The circuit may depend on F and T, but the size bound does not depend on the bit length of T.

Here and throughout, circuits use the full fan-in-two Boolean basis and have unrestricted fan-out. The results also hold over the basis $\{\mathsf{AND}, \mathsf{OR}, \mathsf{NOT}\}$ with a constant-factor change. Circuit size refers to the whole multi-output circuit, not to a separate size bound for each output bit.

Corollary 1.2. *If $\mathrm{C}(F_n) = n^{O(1)}$, then*

$$\sup_{T \geq 0} \mathrm{C}(F_n^T) \leq 2^{n/2 + O(\log n)}. \tag{1.3}$$

The constant hidden in $O(\log n)$ may depend on the polynomial bound for $\mathrm{C}(F_n)$, but not on T.

The quantifiers in Theorem 1.1 are important. For each F and T we construct a circuit that works for *every* initial state. We do not construct a single circuit taking an arbitrary-length exponent as input. Nevertheless, an m-bit exponent can be supplied as input at a multiplicative cost of m:

$$\mathrm{C}((x,t) \mapsto F^t(x)) = O\Big(m\left(A + \sqrt{n^3 NA}\right)\Big), \qquad t \in \{0,1\}^m. \tag{1.4}$$

We prove this in Section 5.1. In particular, $m = n$ preserves the exponent in Equation (1.3). We also give a shared-dictionary construction for r independent initial states, replacing the main square-root term $r\sqrt{n^3 NA}$ of separate constructions by $\sqrt{rn^3 NA}$; see Section 5.2.

For arbitrary fixed powers, the exponent $1/2$ cannot be improved in general. In Section 7 we give an explicit family of permutations P_n with $\mathrm{C}(P_n) = O(n)$ and

$$\max_{T \geq 0} \mathrm{C}(P_n^T) = 2^{n/2 + O(\log n)} \tag{1.5}$$

for even n. The hard exponent is obtained by counting over a very large cyclic group of powers. This is not an explicit exponential circuit lower bound: the permutation is explicit, but the hard power is not specified by an efficient procedure. Moreover, the same permutation has an easy iterator when the time parameter has polynomially many bits.

For a permutation P, choosing $T = \mathrm{ord}(P) - 1$ recovers a circuit for P^{-1}. Thus every polynomial-size permutation on n bits admits an exact inverter of size $2^{n/2 + O(\log n)}$. This exponential upper bound is compatible with one-wayness against polynomial-size circuits. We explain this consequence and its limitations in Section 8.

1.1 Proof overview

The functional graph of F consists of directed cycles with directed trees feeding into them. For a parameter L, we select $O(N/L)$ checkpoints so that every trajectory either encounters one within $2L$ steps or repeats a state within that many steps. A repeated state reveals a period and permits direct recovery of the answer from the initial trajectory.

Set $K = 2L$ and assume $T \geq K$. At each checkpoint h, store $z_h = F^{T-K}(h)$. If the initial trajectory encounters h at time $d \leq K$, then

$$F^{K-d}(z_h) = F^{K-d}\big(F^{T-K}(F^d(x))\big) = F^T(x). \tag{1.6}$$

Thus only a short *forward* correction is needed after the lookup. There is no inverse operation, so the argument applies to noninjective maps.

A separate lookup for each of $\Theta(L)$ visited states against $\Theta(N/L)$ checkpoints would cost $\Theta(N)$ comparisons. Instead, we generate the full initial trajectory before looking anything up and resolve all its queries together. Sorting the queries and dictionary records reduces this cost to $\mathrm{poly}(n)(L + N/L)$ Boolean gates. Combining the initial and final short simulations with this batched lookup gives Equation (1.1).

1.2 Related work and status of the formulation

Preprocessing trajectories to exchange storage for later computation is classical. Hellman's cryptanalytic time–memory tradeoff is an early example of this approach [2]. More directly, Munro, Raman, Raman, and Rao [3, Section 3.1] give a shortcut representation for permutation inversion: with a parameter L, their black-box construction uses $O(L)$ evaluations and $O((N/L)(\log N + \log L))$ auxiliary bits. They also give succinct representations of general functions supporting positive powers with a word-size exponent [3, Section 5]. Those results concern representation space, RAM time, and black-box access, rather than total Boolean gate count.

A closer circuit antecedent is Hirahara, Ilango, and Williams [4, Section 3.2]: they obtain square-root-size permutation-inversion circuits using checkpoints and batched lookup. Their lookup theorem builds on a lemma of Paul. This draft generalizes that permutation-inversion construction to arbitrary fixed forward powers of possibly noninjective maps. It does not subsume their general-function inversion and compression theorems.

Eppstein [5] studies uniform iteration of polynomial-time bijections, with the iteration count supplied in binary, and characterizes the resulting class of functional problems as $\mathsf{FP}^{\mathsf{PSPACE}}$. Our resource question differs: the map is fixed, the circuits are nonuniform, and the size bound remains exponential in the state length. Our result supplies no polynomial-time algorithm for that iteration problem.

The present construction uses the same checkpoint and batched-lookup principles. Batcher's sorting networks [1] implement the lookup, while the shifted payload in Equation (1.6) and the checkpoint-or-repetition lemma handle general powers and functional graphs. No RAM memory probe is treated as a unit-cost circuit operation. Checkpointing, sorting-based lookup, and square-root-size permutation inversion are not claimed as new. Historical priority for the full fixed-power theorem and the shared bounds in this draft remains unresolved.

2 Model and circuit primitives

2.1 Conventions

All logarithms are to base two. The notation F^t denotes t-fold composition, with F^0 equal to the identity. The quantity $\mathrm{C}(G)$ is the minimum number of internal gates in an acyclic Boolean circuit computing all output bits of G. Each internal gate computes a Boolean function of at most two bits. Inputs, wires, fan-out, and output designations are not charged as gates. The two constant bits are available; charging a constant number of gates for them would not change any bound.

The circuit for an iterate is allowed to depend on the entire function F and on its fixed exponent. Preprocessing time and the time required to find the circuit are not bounded. Hardwired data do not provide free addressing: every comparison, multiplexer, and routing operation used to access them is realized with Boolean gates and included in our estimates.

All state words have n bits. Because $L \le 2^n$, indices up to $2L$ fit in $n+2$ bits. In the batched construction $r \le 2^n$, so query identifiers and record positions also have $O(n)$ bits. We retain explicit polynomial factors in n for a concrete implementation; these factors are not claimed to be optimal.

2.2 Arithmetic and selection

Comparison, addition, subtraction, and conditional selection of w-bit words have $O(w)$-gate circuits. Binary long division gives an $O(w^2)$-gate circuit for the remainder of a w-bit integer modulo a positive w-bit integer: process the dividend bits from most to least significant, and at each stage conditionally subtract the modulus from the current shifted remainder. There are $O(w)$ stages and $O(w)$ gates per stage.

A circuit can select one of q words of width w using $O(qw)$ gates by a binary tree of word multiplexers, padding to a power of two when necessary. It can also select one flagged record from an ordered list of q records in $O(qw)$ gates. For example, a left-to-right scan maintains a bit indicating whether a flagged record has already been found and retains the first one. This scan is simply an acyclic chain of gates; no depth bound is needed here.

Lemma 2.1 (Batched fixed-dictionary lookup)**.** *Suppose a fixed dictionary consists of M distinct w-bit keys, each with a w-bit payload. Given $Q \ge 1$ query keys, a Boolean circuit can output a match bit and the corresponding payload for every query, in the original query order, using*

$$O\Big((M+Q)(w+\log(M+Q+2))\log^2(M+Q+2)\Big) \tag{2.1}$$

gates. The payload of an unmatched query can be set to zero. In particular, if $w = O(n)$ and $\log(M+Q+2) = O(n)$, the cost is $O(n^3(M+Q))$.

Proof. Let $R = M+Q$. Form one record for each dictionary entry and one for each query. A record contains a key, a dictionary/query tag, a payload field, and an original-position field. The record width is $O(w+\log(R+2))$.

Sort the records lexicographically by key, then by the tag with dictionary records first, and then by original position. A sorting network with $O(R\log^2(R+2))$ comparators exists [1]. A comparator and the resulting conditional swap of the entire records cost $O(w+\log(R+2))$ Boolean gates. Padding uses a separate dummy-record bit so that no actual key is reserved as a sentinel.

In the sorted list, a dictionary record precedes every query with its key. Scan the list, resetting the match bit when the key changes and propagating the payload from a dictionary record through the equal-key run that follows it. If an equal-key run has no dictionary record, every query in that

run receives match bit zero. Dictionary keys are distinct, so no run contains two competing payloads. This scan uses $O(R(w + \log(R + 2)))$ gates.

Finally, sort records by a tag that places queries first and then by the query's original position. The first Q records give the requested outputs in order. A second sorting network has the same asymptotic cost as the first. Every operation described is a fixed acyclic circuit. □

Lemma 2.2 (Detecting a repeated state)**.** *Given Q words of w bits, a circuit can return a flag indicating whether two words are equal and, when the flag is one, indices $i < j$ of two equal words. Its size is*

$$O\Big(Q(w + \log(Q + 2)) \log^2(Q + 2)\Big).$$

In particular, for $Q \le 2^{n+O(1)}$ and $w = O(n)$, the size is $O(n^3 Q)$.

Proof. Attach each word's original index and sort by word and then by index. A duplicate exists exactly when two adjacent sorted records have the same word. Compare adjacent words and select one flagged pair. Sorting by index within an equal-word run guarantees that the returned indices satisfy $i < j$. The remaining comparisons and selection use at most $O(Q(w + \log(Q + 2)))$ gates. □

3 Sparse checkpoints in functional graphs

The functional graph of a map F has one vertex for each state and the edge $v \to F(v)$ at every vertex. Each weakly connected component has a unique directed cycle, with directed trees feeding into it. For a noncycle vertex, its *depth* is the number of steps needed to reach the cycle.

Lemma 3.1 (Checkpoint or repetition)**.** *Let F be a function on a set of N states, and let $1 \le L \le N$. There is a set H of at most $\lfloor N/L \rfloor$ states such that, for every x, either two of $x, F(x), \dots, F^{L-1}(x)$ are equal or $F^d(x) \in H$ for some $d < 2L$. More generally, the same conclusion holds for starting states in any finite forward-invariant set S, with $|H|L \le |S|$ and $H \subseteq S$. The ambient state type need not be finite.*

Proof. Choose a maximal family of pairwise vertex-disjoint simple directed paths, each consisting of L vertices in S. Put the last vertex of each path into H. Disjointness gives $|H|L \le |S|$. If the first L states of an orbit are distinct, they form such a path in S. By maximality this path meets a chosen path: say $F^i(x) = F^j(a)$, where $i, j < L$ and the chosen path starts at a. Its marked endpoint is reached from x after $i + (L - 1 - j) < 2L$ steps. If the prefix is not simple, it already contains a repetition. Taking S to be the entire state space proves the first claim. The Lean proof implements the same packing argument by induction on $|S|$, removing one simple path at a time. □

Remark 3.2. It is essential that the lemma allows a short repetition instead of a checkpoint. A function can have N fixed points, in which case a set meeting every eventual cycle must contain all N states. Such cycles need no stored shortcut: their behavior is already visible locally.

4 Circuits for fixed powers

We now prove Theorem 1.1. The construction is uniform as a description of a circuit from the chosen constants, but no efficient procedure for computing those constants is asserted.

4.1 Short trajectories and shifted payloads

Fix L and let $K = 2L$. If $T < K$, use T copies of a circuit for F; this costs at most $2Ls$ gates. Hence assume $T \geq K$. Choose H as in Lemma 3.1. For every $h \in H$, hardwire the n-bit payload

$$z_h = F^{T-K}(h). \tag{4.1}$$

The dictionary is fixed independently of the input x and has at most N/L entries.

On input x, first compute

$$x_0 = x, \quad x_1 = F(x), \quad \ldots, \quad x_K = F^K(x). \tag{4.2}$$

This requires Ks gates. Apply Lemma 2.1 to all $K+1$ states in Equation (4.2), using the dictionary $h \mapsto z_h$. Select one matching record, if any, and retain its original trajectory index d. This gives a match flag and a pair (d, z_h) with $x_d = h$ whenever the flag is one.

For the matched case, compute a second short trajectory

$$z_h, F(z_h), \ldots, F^K(z_h)$$

and select its entry at position $K - d$. Its value is

$$\begin{aligned} F^{K-d}(z_h) &= F^{K-d}\big(F^{T-K}(h)\big) \\ &= F^{T-d}(h) \\ &= F^{T-d}\big(F^d(x)\big) = F^T(x). \end{aligned} \tag{4.3}$$

All exponents used here are nonnegative. This second trajectory costs another Ks gates; selection costs $O(nK)$ gates.

4.2 The repeated-state case and very large exponents

Use Lemma 2.2 on Equation (4.2). If no checkpoint was found, Lemma 3.1 guarantees indices

$$0 \leq i < j \leq K, \qquad x_i = x_j.$$

Put $p = j - i$. Determinism implies $x_{u+p} = x_u$ for all $u \geq i$. The number p need not be the least period. Since $T \geq K \geq i$,

$$F^T(x) = x_{\,i+((T-i) \bmod p)}. \tag{4.4}$$

The selected index is at most $i + p - 1 = j - 1$, so the required state already occurs in the initial trajectory.

To implement Equation (4.4) without paying for the bit length of T, hardwire the table

$$R_T(p) = T \bmod p, \qquad 1 \leq p \leq K. \tag{4.5}$$

There are K entries, each of $O(n)$ bits. A multiplexer tree retrieves $R_T(p)$ in $O(nK)$ gates. Compute $i \bmod p$ by binary long division and then compute

$$(T - i) \bmod p = \big(R_T(p) - (i \bmod p)\big) \bmod p$$

using a comparison, subtraction, and a conditional addition of p. These operations take $O(n^2)$ gates. A further $O(nK)$ gates select the state prescribed in Equation (4.4).

A circuit evaluates both branches, so all intermediate operations must also be defined when their output will be discarded. If a repeated pair is absent, supply the dummy values $i = 0$ and $p = 1$ to the arithmetic branch. If a checkpoint is absent, supply a dummy state and $d = 0$ to the second trajectory. The final output selects the checkpoint branch when its match flag is one, and otherwise selects the repetition branch. The checkpoint lemma ensures that the selected branch is always valid. In particular, no division by zero occurs on any input.

4.3 Size bound

The two simulations together use at most $2Ks = 4Ls$ gates. The dictionary and query list contain

$$|H| + (K+1) = O\left(\frac{N}{L} + L\right)$$

records of $O(n)$ bits, and the logarithm of this number is $O(n)$. Thus Lemma 2.1 costs $O(n^3(N/L+L))$ gates. Repeated-state detection costs $O(n^3L)$ gates, and all remaining lookup, arithmetic, and selection operations cost $O(nL + n^2)$ gates. Since $n, L \geq 1$, the latter cost is absorbed by $O(n^3L)$. This proves Equation (1.1).

Write the tradeoff as $O(AL + n^3N/L)$, where $A = s + n^3$, and choose

$$L = \max\left\{1, \left\lceil\sqrt{n^3N/A}\right\rceil\right\}. \tag{4.6}$$

Since $A \geq n^3$, this choice satisfies $L \leq \left\lceil\sqrt{N}\right\rceil \leq N$. If $A \leq n^3N$, then $\sqrt{n^3N/A} \leq L \leq 2\sqrt{n^3N/A}$. Hence

$$AL = O(\sqrt{n^3NA}), \qquad n^3N/L = O(\sqrt{n^3NA}).$$

If $A > n^3N$, the choice is $L = 1$ and the cost is $O(A)$. This establishes Equation (1.2) and completes the proof of Theorem 1.1. □

Remark 4.1 (Where the exponent is stored). For $T \geq K$, the information about T used by the circuit is confined to the checkpoint payloads in Equation (4.1) and the residue table in Equation (4.5). Their total length is $O(n(N/L + L))$ bits. The construction does not place the binary expansion of T on a long hidden input tape, and it does not perform uncharged arithmetic on an arbitrarily large integer. Computing the hardwired information is a preprocessing task outside the asserted circuit bound.

4.4 Invariant sets and image-sensitive bounds

Theorem 4.2 (Invariant-set tradeoff)**.** *Let $S \subseteq \{0,1\}^n$ be forward invariant under F, with $|S| = M$. For every fixed $T \geq 0$ and $1 \leq L \leq N$, there is a circuit agreeing with F^T on S of size*

$$O\left(Ls + n^3\left(L + \frac{M}{L}\right)\right).$$

The circuit is total on $\{0,1\}^n$; correctness outside S is not asserted.

Proof. Use the invariant-set version of Lemma 3.1, and otherwise apply the fixed-power construction unchanged. A state still occupies n bits, but the dictionary contains only $O(M/L)$ records. □

Corollary 4.3 (Image sensitivity)**.** *Put $M_d = |\operatorname{im}(F^d)|$. For $T \geq d$ and $1 \leq L \leq M_d$,*

$$\mathrm{C}(F^T) \leq \mathrm{C}(F^d) + O\left(Ls + n^3\left(L + \frac{M_d}{L}\right)\right).$$

Consequently this is at most $\mathrm{C}(F^d) + O(A + \sqrt{n^3M_dA})$.

Proof. The image of F^d is forward invariant, since $F(F^d(x)) = F^d(F(x))$. Compute $F^d(x)$ and apply Theorem 4.2 on its image for $T - d$ more steps. Optimize L as in Equation (4.6), replacing N by M_d. □

5 Input times and shared evaluation

5.1 An iteration count supplied as input

Definition 5.1. For $m \geq 1$, define

$$\mathrm{Iter}_{F,m} : \{0,1\}^n \times \{0,1\}^m \longrightarrow \{0,1\}^n, \qquad \mathrm{Iter}_{F,m}(x,t) = F^{\mathrm{val}(t)}(x),$$

where $\mathrm{val}(t)$ is the nonnegative integer encoded by t.

Theorem 5.2 (Variable-time iteration)**.** *Under the assumptions of Theorem 1.1, for every $m \geq 1$,*

$$\mathrm{C}(\mathrm{Iter}_{F,m}) = O\Big(m\left(A + \sqrt{n^3 NA}\right)\Big). \tag{5.1}$$

In particular,

$$\mathrm{C}(\mathrm{Iter}_{F,n}) = O\Big(n\left(A + \sqrt{n^3 NA}\right)\Big).$$

If s and m are polynomial in n, then $\mathrm{C}(\mathrm{Iter}_{F,m}) \leq 2^{n/2+O(\log n)}$.

Proof. Write $t = \sum_{j=0}^{m-1} t_j 2^j$. For each j, apply Theorem 1.1 directly to the original map F and the fixed exponent 2^j, obtaining a circuit for $G_j = F^{2^j}$. Starting with $v_0 = x$, use

$$v_{j+1} = \begin{cases} G_j(v_j), & t_j = 1, \\ v_j, & t_j = 0. \end{cases}$$

Inductively, $v_j = F^{\sum_{a<j} t_a 2^a}(x)$, so $v_m = F^t(x)$. There are m circuits of the size in Equation (1.2) and m n-bit multiplexers. The extra $O(mn)$ gates are absorbed by the claimed bound. Each G_j is constructed from F using the theorem; it is not obtained by duplicating the circuit for G_{j-1}. □

5.2 Sharing a dictionary among independent trajectories

Let

$$B_{F,T,r}(x_1, \ldots, x_r) = (F^T(x_1), \ldots, F^T(x_r)).$$

The inputs $x_1, \ldots, x_r$ are independent n-bit words. The following construction uses one checkpoint dictionary for all of them.

Theorem 5.3 (Batched fixed powers)**.** *Under the assumptions of Theorem 1.1, for every $1 \leq r \leq N$, $T \geq 0$, and $1 \leq L \leq N$,*

$$\mathrm{C}(B_{F,T,r}) = O\left(rLs + n^3\left(rL + \frac{N}{L}\right)\right). \tag{5.2}$$

Consequently,

$$\mathrm{C}(B_{F,T,r}) = O\Big(rA + \sqrt{rn^3 NA}\Big). \tag{5.3}$$

Proof. Set $K = 2L$. If $T < K$, unroll T copies of F independently for each input. Otherwise use the checkpoint set and dictionary from Section 4. Generate the first $K+1$ states of all r trajectories. Together with one copy of the dictionary, these give

$$R = |H| + r(K+1) = O(N/L + rL)$$

records. Attach the trajectory number and within-trajectory position to each query. Because $r \leq N$ and $L \leq N$, the record width and $\log(R+2)$ are $O(n)$. One application of Lemma 2.1 therefore resolves all queries in $O(n^3(N/L + rL))$ gates and returns them in their original order.

Select a matching checkpoint separately within each trajectory. Run repeated-state detection separately on each trajectory as well; this costs $O(rn^3L)$ gates and does not confuse a collision between two different trajectories with a repetition along one trajectory. The correction trajectory and output selection are also performed separately for each input. The fixed residue table may be implemented with a separate $O(nL)$-gate lookup for each trajectory, whose total cost is absorbed by $O(rn^3L)$. The two simulation phases together cost $O(rLs)$ gates. This proves Equation (5.2).

Choose

$$L = \max\left\{1, \left\lceil \sqrt{n^3N/(rA)} \right\rceil \right\}.$$

As before $1 \le L \le N$. If $rA \le n^3N$, then $rAL = O(\sqrt{rn^3NA})$ and $n^3N/L = O(\sqrt{rn^3NA})$. If $rA > n^3N$, the choice is $L = 1$ and the cost is $O(rA)$. This gives Equation (5.3). □

Corollary 5.4 (Independent input times)**.** *For $1 \le r \le N$ and $m \ge 1$, the map*

$$\big((x_1, t_1), \ldots, (x_r, t_r)\big) \longmapsto (F^{t_1}(x_1), \ldots, F^{t_r}(x_r)), \qquad t_i \in \{0,1\}^m,$$

has a circuit of size

$$O\Big(m\left(rA + \sqrt{rn^3NA}\right)\Big). \tag{5.4}$$

Proof. Use the binary-power construction of Theorem 5.2. At stage j, apply Theorem 5.3 with update map F and exponent 2^j on the r current states, then use the jth bit of each t_i to select whether to retain or update that state. The $O(mrn)$ selection gates are absorbed by Equation (5.4). □

For polynomial s and m, and $r = \lfloor 2^{\alpha n} \rfloor$ with fixed $0 \le \alpha \le 1$, Equation (5.4) is at most

$$2^{(1+\alpha)n/2 + O(\log n)}.$$

This is an upper-bound sharing statement. It is not a direct-sum separation against optimal circuits for a single trajectory.

6 Computing the Structure of Functional Graphs

Write $B = A + \sqrt{n^3NA}$. For a state x, let $\mu(x)$ be its distance to its eventual cycle, let $\ell(x)$ be that cycle's length, and let $c(x)$ be the lexicographically least state on the cycle. Define $\phi(x) \in \{0, \ldots, \ell(x) - 1\}$ by

$$F^t(x) = F^{(\phi(x)+t) \bmod \ell(x)}(c(x)) \qquad (t \ge \mu(x)). \tag{6.1}$$

The phase is an eventual-orbit label; it does not assert that a transient state itself lies on the cycle. These labels are unique. Cycle lengths are encoded with $n+1$ bits, since a cycle may have length N.

Theorem 6.1 (Cycle labels)**.** *A circuit of size $O(B)$ computes $(\ell(x), c(x), \phi(x))$ for every x.*

Proof. Fix L and use the checkpoint construction with $K = 2L$. Hardwire the three labels for each checkpoint. If $F^d(x) = h$ for $d \le K$, output

$$\ell(x) = \ell(h), \qquad c(x) = c(h), \qquad \phi(x) = (\phi(h) - d) \bmod \ell(h).$$

This follows by applying Equation (6.1) after both transients have ended. All records still have $O(n)$ bits.

If there is no checkpoint hit, the prefix repeats. Sort prefix records by state and then by original index. Among adjacent equal-state pairs, choose the pair with smallest second index j, retaining its first index i. This is the first repetition in the original trajectory: any earliest second occurrence is adjacent to its first occurrence in its equal-state run. The states before j are distinct, so $i = \mu(x)$ and $[i, j)$ consists of exactly one cycle traversal. Set $\ell = j - i$, and select the minimum state $c = x_p$ over $i \le p < j$. Then $\phi(x) = (-p) \bmod \ell$. Minimum selection and the interval mask cost $O(nK)$ gates, and word arithmetic costs $O(n^2)$ gates. Sorting and lookup cost $O(n^3(L + N/L))$, while the prefix costs $O(Ls)$. Optimizing L proves the claim. Discarded branches use safe positive dummy moduli, as in the fixed-power construction. □

Theorem 6.2 (Reachability and first hitting times)**.** *A circuit of size $O(n^2B)$, on input (x, y), decides whether $F^t(x) = y$ for some $t \ge 0$ and returns the least such t when it exists. The output may use a separate existence flag and n time bits.*

Proof. Let $D = \operatorname{lcm}(1, \ldots, N)$. A state z is cyclic if and only if $F^D(z) = z$: a cyclic state has period dividing D, whereas any positive return makes z cyclic. Thus cyclicity has an $O(B)$-gate test by Theorem 1.1. For each x, the predicate "$F^j(x)$ is cyclic" is monotone in j and is true at $j = N - 1$. Binary search on the inclusive interval $[0, N - 1]$ therefore finds $\mu(x)$ in $O(n)$ adaptive stages. Unroll these stages as a circuit. Each uses the n-bit iterator of Theorem 5.2, of size $O(nB)$, followed by the cyclicity test. Compute $\mu(y)$ similarly; the total is $O(n^2B)$.

Compute both sets of cycle labels using Theorem 6.1. If y is transient, it is reachable exactly when

$$\mu(x) \ge \mu(y) \quad \text{and} \quad F^{\mu(x)-\mu(y)}(x) = y.$$

Indeed, transient depth decreases by exactly one per step until the cycle is reached. In this case the displayed exponent is the first hitting time. The variable-time iterator evaluates this final test.

If y is cyclic, it is reachable exactly when $c(x) = c(y)$. On a positive instance its first hitting time is

$$\mu(x) + \big((\phi(y) - \phi(x) - \mu(x)) \bmod \ell(x)\big),$$

where the parenthesized residue lies in $[0, \ell(x) - 1]$. This is the first time on or after entry into the cycle with the required phase. Finally $\mu(x) + \ell(x) \le N$, so every first hitting time is at most $N - 1$. Arithmetic may use $n + 1$ intermediate bits. Return zero in the time field on negative instances. □

Relation to navigation data structures. Function powers and path queries have established data-structure formulations [3, 7]. In particular, nearest labeled successors include reachability of a fixed singleton target. Those results charge representation space and query time in a RAM model. Here the given object is a succinct transition circuit, and every gate accessing a nonuniform table is charged. These differences specify the claim being compared; they do not establish historical novelty of the present bounds. The two theorems in this section have mathematical proofs here but are outside the current Lean formalization.

7 A matching exponent for arbitrary fixed powers

This section gives an explicit linear-size permutation whose collection of powers is large enough that some member has circuit complexity at least $2^{n/2}/\operatorname{poly}(n)$. The counting argument is elementary and does not require an asymptotic estimate for the maximal order of a permutation.

7.1 A permutation with cycles of all lengths up to $2^{n/2} - 1$

Let $n = 2k$ and $B = 2^k - 1$. Write a state as a pair of k-bit integers (a, b). Define

$$P_n(a,b) = \begin{cases} (a, b+1 \bmod a), & 1 \le a \le B \text{ and } 0 \le b < a, \\ (a,b), & \text{otherwise.} \end{cases} \tag{7.1}$$

For each $1 \le a \le B$, the states $(a,0), (a,1), \ldots, (a,a-1)$ form one cycle of length a; every other state is fixed. Hence P_n is a permutation and

$$\operatorname{ord}(P_n) = D_B := \operatorname{lcm}(1, 2, \ldots, B). \tag{7.2}$$

Despite the notation in Equation (7.1), the update does not require a general division circuit. Compare b with a, increment b, and, when $b < a$, replace b by zero exactly when the increment equals a. Otherwise retain the appropriate increment or original value. Comparison, increment, and selection each have $O(k)$-gate circuits. The first coordinate is passed through unchanged. Thus

$$\mathrm{C}(P_n) = O(n), \tag{7.3}$$

and the family has a direct polynomial-time circuit construction.

Lemma 7.1. *For every integer $B \ge 1$,*

$$\operatorname{lcm}(1, 2, \ldots, B) \ge \frac{2^B}{B+1}. \tag{7.4}$$

Proof. We show that every binomial coefficient $\binom{B}{j}$ divides D_B. For a prime p, factorial valuations give

$$\nu_p\binom{B}{j} = \sum_{a \ge 1} \left(\left\lfloor \frac{B}{p^a} \right\rfloor - \left\lfloor \frac{j}{p^a} \right\rfloor - \left\lfloor \frac{B-j}{p^a} \right\rfloor \right).$$

Each summand is zero or one, and it is zero when $p^a > B$. Therefore

$$\nu_p\binom{B}{j} \le \left\lfloor \log_p B \right\rfloor = \nu_p(D_B),$$

where both sides are zero for $p > B$. This proves divisibility. Since $\sum_{j=0}^{B} \binom{B}{j} = 2^B$, some binomial coefficient is at least $2^B/(B+1)$, and D_B is at least that large. □

Lemma 7.2 (Counting multi-output circuits)**.** *The number of functions $\{0,1\}^n \to \{0,1\}^n$ computable with at most q fan-in-two Boolean gates is at most*

$$(q+1)16^q(n+q+2)^{2q+n} = 2^{O((q+n)\log(n+q+2))}. \tag{7.5}$$

Proof. Topologically order the gates. At each gate choose one of at most 16 Boolean operations and two predecessors among the inputs, the two constants, and gates. Allowing all $n+q+2$ possible predecessors at each position only increases the count. Choose the n output wires from these same possibilities. Sum over gate counts from zero through q; each summand is bounded by the expression without its factor $q+1$. Different encodings may compute the same function, which only helps the upper bound. □

Theorem 7.3 (Hard powers of a simple permutation)**.** *For all sufficiently large even n, the permutation in Equation* (7.1) *satisfies*

$$\max_{T\geq 0} \mathrm{C}(P_n^T) = \Omega\left(\frac{2^{n/2}}{n}\right). \tag{7.6}$$

Together with Theorem 1.1, this gives

$$\log_2\left(\max_{T\geq 0} \mathrm{C}(P_n^T)\right) = \frac{n}{2} + O(\log n). \tag{7.7}$$

Proof. There are exactly D_B distinct powers of P_n, with representatives $0 \leq T < D_B$. By Lemma 7.1,

$$\log_2 D_B \geq B - \log_2(B+1).$$

Let $q = \lfloor cB/n \rfloor$ for a sufficiently small positive absolute constant c. Since $B = 2^{n/2} - 1$, Lemma 7.2 bounds the logarithm of the number of circuits of size at most q by

$$O((q+n)\log(n+q+2)) = O(cB + n^2).$$

Choose c so that the coefficient of B is less than $1/2$. For all sufficiently large even n, this quantity is smaller than $B - \log_2(B+1)$. Thus there are fewer size-q circuits than distinct powers of P_n. At least one power has complexity greater than q, proving Equation (7.6).

For the upper bound, substitute $\mathrm{C}(P_n) = O(n)$ into Equation (1.2). This gives $\max_T \mathrm{C}(P_n^T) = O(n^3 2^{n/2})$. Combining the upper and lower bounds and taking logarithms proves Equation (7.7). □

Corollary 7.4. *For all sufficiently large even n, some power of P_n has an output coordinate of circuit complexity $\Omega(2^{n/2}/n^2)$.*

Proof. For the power supplied by Theorem 7.3, join optimal circuits for all n output coordinates. Their total size is at most n times the largest coordinate complexity and is an upper bound on the complexity of the whole map. □

7.2 Why the lower bound does not apply to short input times

The lower bound selects T from $\{0, \ldots, D_B - 1\}$; it places no polynomial bound on the bit length of that exponent. Indeed, Lemma 7.1 shows that $\log D_B \geq 2^{n/2} - O(n)$. The choice of a power therefore has exponentially many available bits. The theorem does not provide a polynomial-time procedure that, given n, identifies a hard power.

For the very same family, the iterator with an m-bit time input has size $O(nm + n^2)$. On an active cycle,

$$P_n^t(a, b) = (a, (b + (t \bmod a)) \bmod a). \tag{7.8}$$

An m-stage remainder computation with a k-bit running remainder costs $O(mk)$ gates, and the remaining comparison, addition, and selection cost at most $O(n^2)$ gates. On the inactive states the answer is the input itself; use a dummy positive modulus when $a = 0$. Consequently, for $m = \mathrm{poly}(n)$ this iterator has polynomial size.

Corollary 7.5 (Bit length of hard exponents)**.** *Let T_n be exponents satisfying $\mathrm{C}(P_n^{T_n}) = \Omega(2^{n/2}/n)$ for sufficiently large even n. Their binary lengths $m_n = \max\{1, \lceil \log_2(T_n + 1) \rceil\}$ satisfy*

$$m_n = \Omega(2^{n/2}/n^2).$$

Proof. Hardwire the m_n-bit exponent into the iterator just described. This gives $\mathrm{C}(P_n^{T_n}) = O(nm_n + n^2)$. Comparing with the assumed lower bound and using $n^2 = o(2^{n/2}/n)$ proves the claim. □

Thus the hard powers necessarily require exponentially long binary exponents; long descriptions are not merely allowed by the counting proof. Theorem 7.3 establishes sharpness for arbitrary fixed powers. It gives no hardness result for the bounded-bit-time problem.

8 Permutation inversion and one-way functions

8.1 Inversion as a fixed power

Corollary 8.1 (Exact permutation inversion)**.** *Let $P : \{0,1\}^n \to \{0,1\}^n$ be a permutation with $\mathrm{C}(P) \le s$, and put $N = 2^n$ and $A = s + n^3$. For every $1 \le L \le N$,*

$$\mathrm{C}(P^{-1}) = O\Big(Ls + n^3\left(L + \frac{N}{L}\right)\Big).$$

In particular,

$$\mathrm{C}(P^{-1}) = O\Big(A + \sqrt{n^3 NA}\Big).$$

If $s = n^{O(1)}$, an inverter of size $2^{n/2+O(\log n)}$ is correct on every input.

Proof. Let $D = \mathrm{ord}(P)$, the least common multiple of its cycle lengths. Then P^D is the identity, so $P^{-1} = P^{D-1}$. Apply Theorem 1.1. The exponent may have a long binary expansion, but the theorem charges only the resulting tables and their access gates. □

This recovers the permutation-inversion circuit bound already discussed by Hirahara, Ilango, and Williams [4, Section 3.2]. It identifies an earlier special case of the fixed-power theorem.

8.2 Compatibility with one-wayness

A cryptographic one-way function is computable in polynomial time, while every probabilistic polynomial-time inverter succeeds with only negligible probability on the image of a uniform input. One may instead require security against every polynomial-size circuit family, a stronger nonuniform notion [6, Section 7.1.1]. For a length-preserving family $f_n : \{0,1\}^n \to \{0,1\}^n$, the latter requires that, for every such family C_n,

$$\Pr_{x \leftarrow \{0,1\}^n}\big[f_n(C_n(f_n(x))) = f_n(x)\big] \tag{8.1}$$

be negligible in n: smaller than n^{-c} eventually for every fixed $c > 0$. Any preimage counts as success. Deterministic circuits suffice in this definition, since random coins can be fixed to maximize average success. A one-way permutation is the bijective special case.

The guarantee in Corollary 8.1 is an exponential size upper bound. It therefore does not contradict either polynomial-time or polynomial-size one-wayness. Nonuniformity alone is not the reason for compatibility: Equation (8.1) already allows arbitrary nonuniform advice encoded in polynomial-size circuits. The relevant distinction is the circuit size. Moreover, no efficient procedure for constructing our inverter is asserted.

There is a quantitative consequence. For every fixed $\varepsilon > 0$, every polynomial-size permutation family on n bits has exact inverters of size at most $2^{(1/2+\varepsilon)n}$ for all sufficiently large n. Thus such a

family cannot remain one-way against all circuits of that size, with any success bound strictly below one. This upper bound leaves open security against polynomial-size circuits, and does not exclude security against size 2^{cn} for a fixed $0 < c < 1/2$. It does not establish that either kind of security is achievable.

8.3 General functions and the counting example

For a noninjective map, inversion is not generally a forward power. For example, let f on $\{0,1,2,3\}$ have values $(0,0,1,0)$. The unique preimage of 1 is 2, whereas every forward iterate of 1 belongs to $\{0,1\}$. Thus Theorem 1.1 does not directly provide square-root-size inverters for arbitrary one-way functions. For comparison, Hirahara, Ilango, and Williams prove a general inversion bound of $2^{4n/5}\,\mathrm{poly}(s+n)$ gates [4, Theorem 3.5].

Encoding exhaustive preimage search as iteration does not avoid the state accounting. The straightforward transition retains an n-bit target y, an n-bit candidate x, and a flag; it tests $f(x) = y$ and either halts or increments x. This uses $2n + O(1)$ state bits. For polynomial-size f, our theorem then gives size $2^{n+O(\log n)}$, not the half exponent measured in the preimage length.

The counting construction in Section 7 also does not establish one-way functions. Its update permutation P_n has an $O(n)$-gate inverse: on an active cycle decrement b modulo a, and leave inactive states fixed. A family of the selected hard powers has exponential worst-case *forward* circuit complexity, so it cannot be a polynomial-time one-way function. The counting proof supplies no average-case inversion hardness. The results here consequently neither prove nor refute the existence of one-way functions or one-way permutations.

9 Lean formalization

The accompanying `lean/` project checks the fixed-power tradeoff and its invariant-set and image-sensitive extensions in Lean 4. Its public statement file imports only CSLib and Mathlib. The proofs use CSLib's circuits and simultaneous synthesis relation, together with the composition, iteration, and verified sorting-network semantics in the author's Circuit library as distributed through `complexitylib` [8, 9, 10]. The project pins Lean and all three libraries; `lean/README.md` gives exact revisions and commands.

Theorem 9.1 (Mechanized tradeoff)**.** *Let c be a circuit over the Boolean constant, NOT, AND, and OR basis, with n inputs and n outputs, and let $S \subseteq \{0,1\}^n$ be finite and invariant under its evaluation F. For all natural numbers T and $1 \le L \le 2^n$, Lean proves the existence of a circuit d agreeing with F^T on S such that*

$$|d| \le 40000 \left[L|c| + (n+1)^3 \left(L + \lfloor |S|/L \rfloor + 1 \right) \right].$$

Here circuit size counts every gate, including constant gates.

The harmless +1s cover zero-width and empty-set cases. For $n, L \ge 1$ the displayed expression implies the paper's asymptotic tradeoff; the real square-root optimization is the argument already given above, not a separate checked Lean theorem. The constant 40000 is a deliberately loose explicit accounting bound. The formal result counts actual Boolean gates and is not conditional on an assumed lookup compiler. Free wires and arbitrary fan-out permit sharing; the two Boolean constants are generated once. Translating the paper's fixed fan-in-two basis to this basis changes size by an absolute constant factor.

Proof organization. `Statement.lean` states the claims, `Proofs/` contains their proofs, and `Main.lean` imports the public results and guards their axiom dependencies. The proof uses the path-packing version of Lemma 3.1. It retains the complete short orbit, with one copy of c per transition.

For the fixed-power circuit it suffices to find one useful collision. The mechanized implementation sorts a single array containing checkpoint records, prefix records, and explicitly tagged padding. Its detector selects adjacent records with equal state keys, unequal complete records, and no padding. Such a pair supplies either a checkpoint hit or two distinct prefix positions with the same state. A sorted equal-key run containing two distinct records has an adjacent unequal pair, which is proved explicitly; identical records cannot create a spurious witness. This avoids needing the full multiquery dictionary interface in the formal proof. The shared comparator computes its comparison once and uses it for every payload bit. Its Batcher sorting bound is $O(rR\log^2 R)$ for R records of r bits.

For a repeated pair $i < j$, put $p = j - i$. Starting at the hardwired residue T mod p, the checked arithmetic adds p while the current value is below i. At most K steps suffice, since $i \le K$; the resulting index lies in $[i, j)$ and is congruent to T modulo p. Words have width $n + 2$ when $K = 2L$, and absence of overflow on valid witnesses is proved. This uses $O(Kn^2)$ gates instead of formalizing variable division, which fits within the asserted $O(n^3L)$ budget. Every branch is total, even when its output is discarded. No operation on the full binary expansion of T is performed by the resulting circuit.

Universes and representations. The packing and reconstruction lemmas quantify over $\alpha : \mathrm{Type}\ u$ for arbitrary u; only the relevant invariant subset must be finite. The theorem `fixed_power_encoded` transports the circuit bound to a finite invariant set $S \subseteq \alpha$ with an encoding $e : \alpha \to \{0,1\}^n$ injective on S, provided $c(e(x)) = e(F(x))$ for $x \in S$. The result computes $e(F^T(x))$ on encoded starting states and counts $|S|$, retaining the actual bit width n. No unit-cost primitive for an arbitrary operation on α is introduced. Separate initialization and decoding circuits, when needed, must be charged separately.

Checked boundary. The fixed-power and invariant-set claims are exported as `fixedPowerBound` and `fixedPowerOnBound`. The encoded-state and image-sensitive claims are `fixed_power_encoded` and `image_sensitive`. All four are checked end to end, including circuit existence, semantics, and size. Their guarded axiom lists contain only `propext`, `Classical.choice`, and `Quot.sound`. There are no `sorry` placeholders or additional axioms. Noncomputable choices select checkpoints, enumeration orders, and circuit witnesses; this is an existence formalization, not an extracted efficient compiler. The navigation theorems, variable-time and batched bounds, counting lower bound, and additional proposals are not claimed to be mechanized. Nor does kernel checking establish historical priority or that the formal statement captures every possible intended circuit convention.

10 Scope and remaining questions

The main theorem is an exact nonuniform upper bound for unrestricted bounded-fan-in circuits. It applies to noninvertible maps and is uniform in the starting state, but not in the circuit construction or its preprocessing cost. It gives no depth bound and no efficient procedure for discovering checkpoints or their payloads from a succinct representation of F. An initial-state-dependent family of update rules must be encoded as a single common transition map before the theorem applies; any additional persistent state changes n and hence the exponential scale of the bound.

The sharpness result deliberately separates the simplicity of an update rule from the description complexity of a chosen power. A stronger lower bound for a specified iteration count, or for an iterator with an n-bit time input, is not established here. The explicit permutation used for counting cannot witness such a strengthening, by Equation (7.8) and Corollary 7.5. The inversion consequence in Corollary 8.1 likewise leaves polynomial-size one-wayness open.

Two further questions remain distinct. First, can one improve the upper bound for short exponent descriptions, or exploit structure beyond the invariant-set reduction above? Second, under what conditions can the required preprocessing be performed substantially faster than exploring the full state space? These questions remain open here. The central conclusion is the unconditional total-gate tradeoff: a short transition circuit can be combined with a sparse table, whose access cost is explicitly accounted for, to compute every fixed iterate at square-root state-space scale.

Provenance and draft status

This draft and the underlying argument were developed with assistance from ChatGPT (GPT-6 Astra Pro), including assistance with theorem formulation, proof development, exposition, and literature searches. The Lean development and the current revision also received assistance from Codex, including circuit synthesis proofs, axiom auditing, and review. The author field is provisional. Historical priority and final attribution require author review; the draft does not represent independent peer verification.

References

[1] Kenneth E. Batcher. Sorting networks and their applications. In *Proceedings of the April 30–May 2, 1968, Spring Joint Computer Conference*, AFIPS Conference Proceedings, volume 32, pages 307–314, 1968. doi:10.1145/1468075.1468121.

[2] Martin E. Hellman. A cryptanalytic time–memory trade-off. *IEEE Transactions on Information Theory*, 26(4):401–406, 1980. Author-hosted full text.

[3] J. Ian Munro, Rajeev Raman, Venkatesh Raman, and S. Srinivasa Rao. Succinct representations of permutations and functions. *Theoretical Computer Science*, 438:74–88, 2012. Preprint: arXiv:1108.1983, 2011.

[4] Shuichi Hirahara, Rahul Ilango, and R. Ryan Williams. Beating brute force for compression problems. In *Proceedings of the 56th Annual ACM Symposium on Theory of Computing (STOC)*, pages 659–670, 2024. doi:10.1145/3618260.3649778. Full version: ECCC TR23-171; section and theorem numbers refer to that version.

[5] David Eppstein. The complexity of iterated reversible computation. *TheoretiCS*, 2, Article 10, 1–41, 2023. doi:10.46298/theoretics.23.10.

[6] Oded Goldreich. *Computational Complexity: A Conceptual Perspective*. Cambridge University Press, 2008. Author's draft, Chapter 7: one-way function definitions.

[7] Travis Gagie, Meng He, and Gonzalo Navarro. Path queries on functions. *Theoretical Computer Science*, 770:34–50, 2019. doi:10.1016/j.tcs.2018.10.021. Author-hosted full text.

[8] The CSLib contributors. CSLib: a Lean library for computer science. `https://github.com/leanprover/cslib`. Pinned revisions are in the accompanying Lean dependency manifest.

[9] The mathlib community. The Lean mathematical library. In *Proceedings of CPP*, pages 367–381, 2020. doi:10.1145/3372885.3373824. Lean 4 implementation: `https://github.com/leanprover-community/mathlib4`.

[10] Samuel Schlesinger and contributors. Complexitylib, including the Circuit library and verified sorting networks. `https://github.com/SamuelSchlesinger/complexitylib`.